

The Weather Trigger — What It Is and How It Works

Status as of July 22, 2026 · Live at predictions.usesmpt.com/triggers · Code:

github.com/mrglenn64/my-predictions (src/weather_trigger/) · Read-only paper mode: no orders, no wallet, no keys — a measurement instrument, not a trading bot.

The idea in one paragraph

Polymarket runs daily markets like 'Highest temperature in Beijing on July 22?', split into buckets (26°C, 27°C, 28°C or above...). A day's maximum temperature can only RISE. So the moment the official station touches 28°C, two things become permanent facts, not forecasts: every bucket below 28 is dead, and '28 or above' has already won. If the market still prices a dead bucket at 20 cents, that gap is not a prediction — it is a mispricing of an already-published fact. This tool watches for exactly those moments across ~30 cities worldwide, around the clock, and measures how long the crowd takes to catch up and how many real dollars sat in the gap.

Why this might work (and where it won't)

- Nobody stares at weather-station feeds for 30 cities in every timezone. Software does that for free. This is the classic 'AI as intern' play: the machine watches one painfully specific condition at inhuman scale and hands the decision to you.
- Polymarket does not ban or limit winners (unlike sportsbooks) — the constraint here is thin order books, not account risk.
- The big cities (Beijing does \$250k/day) likely have professional weather traders already. The realistic hunting ground is smaller cities and the minutes right after a boundary is crossed.

How resolution actually works — the detail everything depends on

Each market's own rules name a SPECIFIC weather station, via a Wunderground link: NYC resolves at LaGuardia airport, Los Angeles at LAX, Seoul at Incheon (40 km from the city), in whole degrees. Forecasting 'the city' is meaningless — LAX runs ~10°F cooler than downtown LA in summer. So the tool parses every market's rules text and extracts the station's airport code (ICAO) from the Wunderground URL. If the rules can't be parsed unambiguously, the market is EXCLUDED and never watched (example: Hong Kong resolves on the Hong Kong Observatory, not an airport — excluded). A human-readable review file of parsed rules is generated before any watching begins.

The machine, step by step

- 1 · RULES PARSER — reads each live market's description; extracts station ICAO, unit (°F/°C), source, measurement window. Unparseable = excluded, with the reason logged.
- 2 · OBSERVATIONS — free global METAR feed (aviationweather.gov) for the exact settlement station. Tracks the running daily maximum since the station's LOCAL midnight (timezone/DST handled properly), including the 6-hourly max groups US stations publish.
- 3 · LOCK DETECTOR — mechanical facts only. DEAD: the observed max already exceeds a bucket's top. PROVEN: an 'X or above' bucket whose floor was already reached. A safety margin (1°F / 0.5°C) absorbs rounding differences between METAR and Wunderground. 'The peak has probably passed' is a forecast and is deliberately NOT flagged.

- 4 · MARKET SNAPSHOT — at every lock and every poll after: best bid/ask and full order-book depth for the certain side, from Polymarket's public book. Headline metric: EDGE-DOLLARS — how much could actually be filled at prices better than the certain value, walking the book. A 40-cent mispricing on \$12 of depth is a toy; only dollars say so.
- 5 · LAG TIMELINE — append-only log: LOCK (fact detected) → SNAPSHOT (each poll while mispriced) → CONCEDE (price crosses 95¢/5¢). The lag between LOCK and CONCEDE is the entire edge, measured.

The five traps it had to survive (each found the hard way)

- Wrong place — forecasting city gridpoints instead of the resolution airports made the model look broken (fixed: parse the station from the rules).
- Wrong market — 'lowest temperature' markets judged by daily-MAX logic produced fake 'dead' flags on overnight-low buckets (fixed: highest-temperature markets only, v1).
- Wrong day — tomorrow's buckets judged by today's observations produced fake locks priced at 35¢ (fixed: the market's date must equal the station's current local day).
- Invisible markets — Polymarket stamps weather events as 'ending' at noon UTC though they trade to midnight; a naive filter hid today's markets every afternoon, exactly the window that matters (fixed: 36-hour lookback).
- Phantom prices — displayed mid-prices can be stale on thin books; every dollar figure comes from walking the real order book instead.

What it is NOT

- Not a trading bot: it never places orders and holds no keys or funds.
- Not a forecast: it only acts on observations that already happened.
- Not yet an edge: a single snapshot showed ~\$100k 'fillable' across 98 locks — a number NOT to be believed until lag is measured over days, because deep-dead buckets are usually already priced correctly and the dollars concentrate on knife-edge buckets with rounding risk.

How to read the results page

predictions.usesmpt.com/triggers shows per-city verdict lines in plain English, e.g. 'Beijing: 6 locks, median lag 22m, median \$14 fillable — real lag, toy depth.' Plus a global line that estimates, honestly, the ceiling of what this trigger would have paid per week — labeled a ceiling, never a forecast. Raw data: /api/triggers.

The decision rule

Run unattended for several days. Hand-audit ~10 locks against Wunderground's own history pages. Then one number decides: median lag × median edge-dollars on same-day locks. If minutes of real lag and meaningful depth exist in the smaller cities, this is a genuine, mechanical income lane and the buying discipline (already built for the crypto lane: real ask, depth check, size caps) gets pointed at it. If the crowd concedes within a poll or the depth is toy — the file closes, in writing, having cost nothing.

Operations

```
pm2 process 'weather-trigger' on kv8 (auto-restarts daily)
python scan.py weather-rules          # regenerate the rules review file
python scan.py weather-trigger 60     # run locally for an hour
exports/weather_rules_review.md       # the parsed-rules audit gate
```

